KEC Platform: Future Work & Implementation Roadmap

This document outlines the strategic roadmap for Krishna Engineering College's digital infrastructure. It consolidates the complete Backend Architecture Plan, the Advanced Admin Panel Control System, and immediate Frontend/UX Improvements into a single, actionable execution strategy.

Part 1: Backend Architecture & Structural Plan

The foundation of the platform will be rebuilt utilizing **Go (Golang)** and **PostgreSQL** to handle concurrent application traffic securely.

1.1 Technology Stack

- **Language:** Go (Golang) 1.21+
- **Framework:** Gin (Lightweight HTTP routing framework)
- **Database:** PostgreSQL with GORM (Go Object-Relational Mapper)
- **Authentication:** JWT (JSON Web Tokens)
- **Storage:** AWS S3 (for notices, events, and gallery media)

1.2 Database Core Schema

- **Application Models:** Capturing schema fields for `SUPER_40`, `EV_COURSE`, `DRONE_COURSE` including full validation for grades, email, and semester structures.
- **Content Models:** Dynamic tables configuring `Announcement`, `Department Updates`, and `Event` features.
- **Authentication Models:** Tracking `AdminUser` profiles securely through bcrypt hashes.

1.3 Key API Endpoints

- **Public:** `POST /api/v1/applications` (Student form submissions) & `GET /api/v1/content/*` (Fetching dynamic data for React).
- **Protected (Admin):** Endpoint suites nested under `/api/v1/admin/*` locked via JWT middleware to handle application CSV parsing and Notice updates.

Part 2: Advanced Admin Panel (Control Center)

The Admin Panel will serve as the Command Center, fully detaching website content reliance from code-level modifications.

2.1 Lead & Admission Management (CRM)

- **Unified Application Board:** An interface mapping incoming submissions. Admins can view individual students, attach internal notes ("Called, requested callback"), and track student progress.
- **Status Workflows:** Easily update a student from `PENDING` to `APPROVED`, automatically triggering acceptance emails via backend webhooks.
- **Bulk Export Tool (CSV):** Seamlessly filter and download large batches of student data into Excel natively from the portal.

2.2 Dynamic Content Management System (CMS)

- **Homepage Configurator:** Modify hardcoded data directly. If *Mechanical Engineering* increases its placement rate to 96%, an admin updates the input field and it goes live instantly.

- **Media / Notice Publisher:** Full CRUD (Create, Read, Update, Delete) capability to upload new notices, complete with PDF/Image attachment capabilities. Add an `is_featured` toggle to dynamically pin critical news.
- **Gallery & Event Albums:** Bulk upload controls allowing marketing staff to post photos immediately post-event.

2.3 Maker-Checker Automation & Security (RBAC)

- Implement Draft/Approve workflow hierarchies. Junior "Editor" roles can draft a Notice, but only a "Super Admin" can publish it. System-wide audit logs will track any configuration changes back to the respective admin account.

Part 3: Frontend Improvements & Technical Stabilization

While the backend is orchestrated, the React frontend must be stabilized securely and optimized for incoming heavy traffic loads to guarantee a 100/100 Lighthouse performance score.

3.1 Performance & Payload Optimization

- **React.Lazy Code Splitting:** Currently, `App.jsx` eagerly imports all 30 pages simultaneously resulting in a massive 1.2 MB initial loading payload. Implementing Router-level lazy loading will drop initial load burdens by over 60%, drastically improving mobile speeds and SEO.

3.2 Eradicating Critical Rendering Bugs

- **State Mutation Integrity:** Refactoring the `Super40ExamQuestions.jsx` logic structure. Eradicating state variable updates executing inside of active `setTimeLeft` timers to prevent `"updating during rendering"` application crashes.
- **Validating Component Modals:** Stripping illegally nested HTML structures (e.g. Anchor tags shoved inside motion Button elements) to prevent accessibility screen readers from breaking and restoring mobile tap precision.
- **Form Completions:** Reinstating the missing **Grade field** input string within `Super40Form.jsx` that was previously mapped to active state but missing visually.
- **Hash Jump Removal:** Fixing dummy `#` hash references across top navigation layouts which fundamentally break `Lenis` smooth-scroll listeners when accidentally clicked.

3.3 Strict Deployment Hygiene

- Utilizing automated tree-shaking methodologies and Linter passes to safely erase over 60 actively compiling ESLint unused variable errors prior to production hosting integrations (Vercel/AWS).